
深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 实验二：分治法——最近点对问题

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 刘刚

报告人： 汉雨 学号： 2024040042

实验时间： 2025年4月20日—2025年4月25日

实验报告提交时间： 2025年4月26日

教务部制

一、实验目的：

- (1) 掌握分治法思想。
- (2) 学会最近点对问题求解方法。

二、实验内容：

- 1、 对于平面上给定的N个点，给出所有点对的最短距离，即，输入是平面上的N个点，输出是N点中具有最短距离的两点。
- 2、 要求随机生成N个点的平面坐标，应用蛮力法编程计算出所有点对的最短距离。
- 3、 要求随机生成N个点的平面坐标，应用分治法编程计算出所有点对的最短距离。
- 4、 分别对N=100000—1000000，统计算法运行时间，比较理论效率与实测效率的差异，同时对蛮力法和分治法的算法效率进行分析和比较。
- 5、 如果能将算法执行过程利用图形界面输出，可获加分。

三、算法原理：

蛮力法原理

作为最容易想到的求最近点对的方法。具体过程为：首先输入平面上的 N 个点，并初始化最小距离为一个较大的值（例如将他设置为 100000000）；然后通过两层循环枚举所有不同的点对，即对每个点 i ，与其后所有点 j ($j > i$) 组成点对，此处需要进行运算的次数为：对于每一对点，计算它们之间的欧几里得距离，并将该距离与当前记录的最小距离进行比较，如果更小则更新最小距离；最终，当所有点对都比较完成后，得到的最小距离即为最近点对的距离。该方法需要比较所有点对，其时间复杂度为 $O(N^2)$ 。

蛮力法时间复杂度分析：

蛮力法的时间复杂度是 $O(n^2)$ 。因为它要把所有点对都检查一遍。假设有 n 个点，那么需要计算的点对数量是

$$C_n^2 = \frac{n(n-1)}{2}$$

所以距离计算次数随 n^2 增长，忽略常数和低阶项后，时间复杂度就是

$$O(n^2)$$

蛮力法伪代码：

Algorithm BruteForceClosestPair($P[1..n]$)

Input: 平面上 n 个点的集合 P

Output: 最近点对及其最小距离 d

1. $d \leftarrow +\infty$

```

2. p1 ← null, p2 ← null
3. for i ← 1 to n-1 do
4.     for j ← i+1 to n do
5.         dist ← Distance(P[i], P[j])
6.         if dist < d then
7.             d ← dist
8.             p1 ← P[i]
9.             p2 ← P[j]
10.        end if
11.    end for
12. end for
13. return (p1, p2, d)

```

分治法原理

在这个问题当中，我们要拆分数据非常简单，只需要按照 x 轴坐标对所有点进行排序，然后选择中点进行分割即可，分割之后我们得到的结果如下：

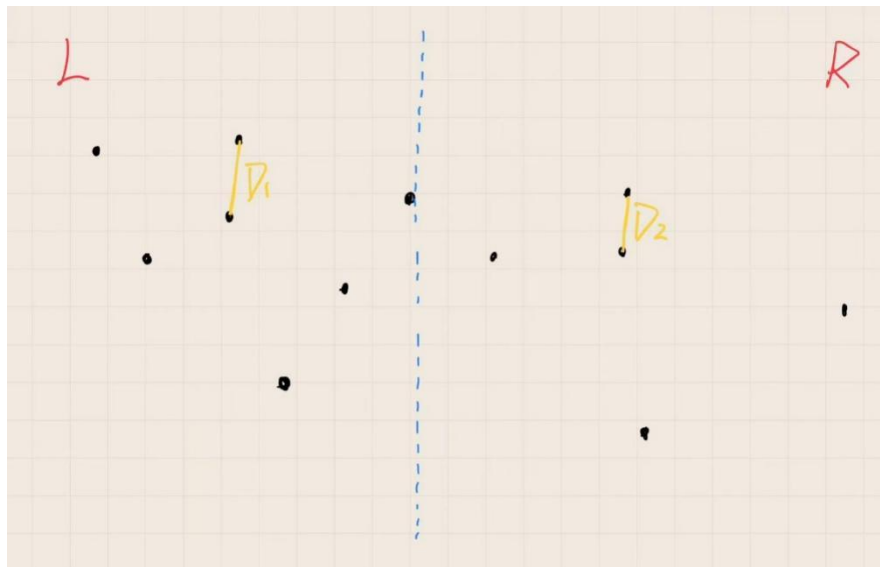


图 1

拆分结束之后，我们只需要分别统计左边部分的最近点对、右边部分的最近点对，以及一个点在左边一个点在右边的最近点对即可，即图中的 $D1$ 和 $D2$

接下来就是需要讨论本算法最重要的一个部分：中间的区域怎么办（合并代价）。

通过递归调用可以获得左边部分 SL 的最短距离 $D1$ 以及右边部分 SR 的最短距离 $D2$ 。再取 $D1$ 和 $D2$ 的较小值，设为 D 。求出了 D 之后，我们就可以用它来限定一个点在 SL 一个点在 SR 这种情况的点队的范围了。

在左侧随便选择一个点 p ，我们来想一个问题，对于点 p 而言， SR 一侧所有的点都有可能与它构成最近点对吗？当然不是，有一些离得远的是明显不可能的，对于这些点我们没有必要一一遍历，直接都可以批量忽略。要想和 p 点构成最近点对，必须在下图这个虚线框起来的范围内。虚线长方形宽为 D ，长为 $2D$ （不将他设为半圆是因为圆形的计算量较大）

事实上左侧选择的点 p 也并非任意的：我们可以讲虚线长方形想象成有关滑动窗口，顺着蓝色分界线滑动，被这个滑动窗口包括进去的点才是我们需要考虑的 p 点。

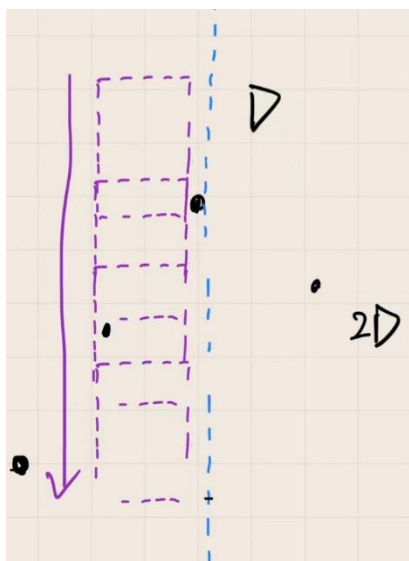


图 2

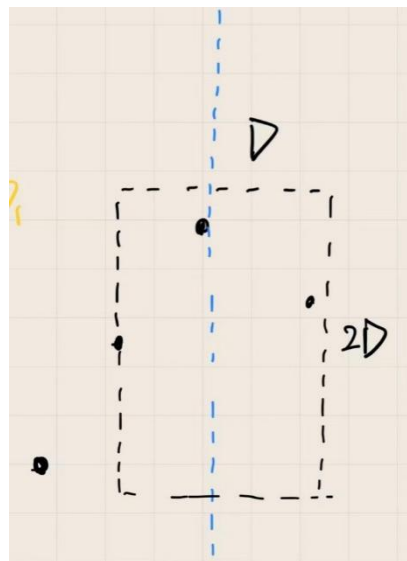


图 3

鸽笼定理在此问题中的应用：

有了这个框后，我们可能会产生一个疑问：会不会有一个极端情况使得这个框内的点会有很多很多。

答案是不可能的。

因为对于框里的点我们有一个基本的要求，就是在这个框里并且在 SR 区域内的点，两两之间的距离不得小于 D 。如果小于 D 了就和我们的结论矛盾了。那么图中的情况其实是不可能的，因为这么多点聚集在一起明显存在两个点的距离小于 D 了，这就矛盾了。

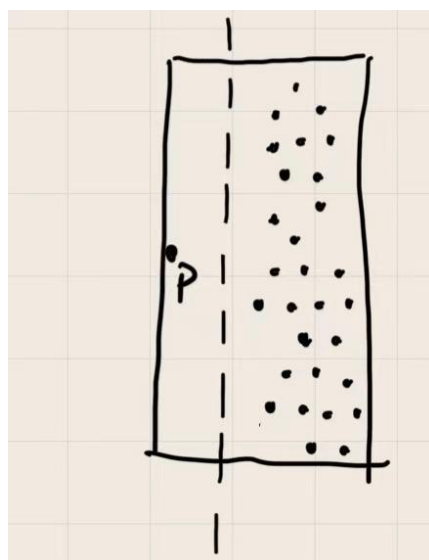


图 4

那么可以继续探究这个方框内最多可以有多少个点：

我们可以先将长方形分成 8 个格子，每个小格子内最多放 1 个点，（若硬塞 2 个点，

他们之间最大的距离是 $\frac{\sqrt{2}}{2}D < D$ ，因此最多一个点）。

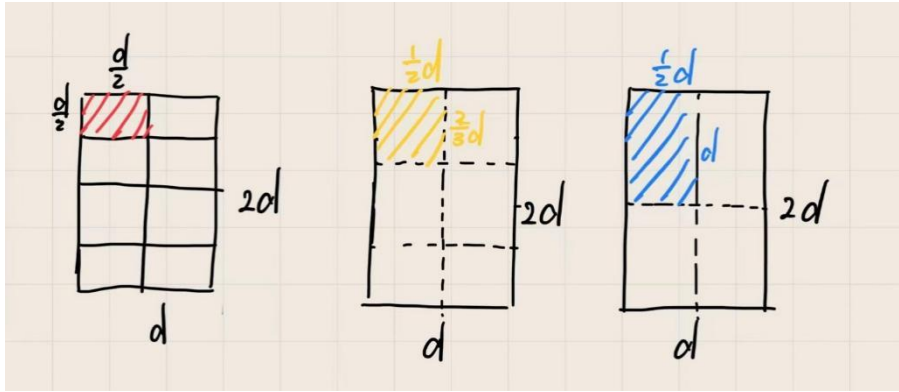


图 5

为了进一步优化算法，我们可以尝试检测比 8 要少的点。因此我们可以将格子分为 6 份，同理发现每个小个子内只能容纳 1 个点，符合鸽笼。

而格子数再往下减少的时候，就会出现对角线大于 D 的情况，这样鸽笼原理就失效了。

因此：鸽笼原理在这道题的关键就是：“每个格子的最大直径必须小于当前最小距离 d ”。如果分成较大的格子（如 $d \times d$ ），其对角线超过 d ，就无法保证同一格内点的距离小于 d ，从而鸽笼原理无法成立。因此必须将区域划分为更小的格子（如 $d/2 \times d/2$ ），才能建立正确的约束关系。

除此之外，我们还可以讲这个长方形的搜索框变成一个边长为 D 的正方形。这是怎么实现的呢？

在条带扫描过程中，对于当前点 p ，仅考虑与其位于中线**相对侧区域**的候选点，而忽略与其处于同侧区域的点对比较。具体而言，当点 p 位于左子区域时，仅检查条带中位于右子区域的候选点；反之亦然。

此外，在扫描顺序上仍保持按 y 坐标有序，并采用单向扫描策略（如自上而下），当候选点与当前点的纵坐标差不小于 D 时立即停止搜索，从而避免回头操作和不必要的比较。

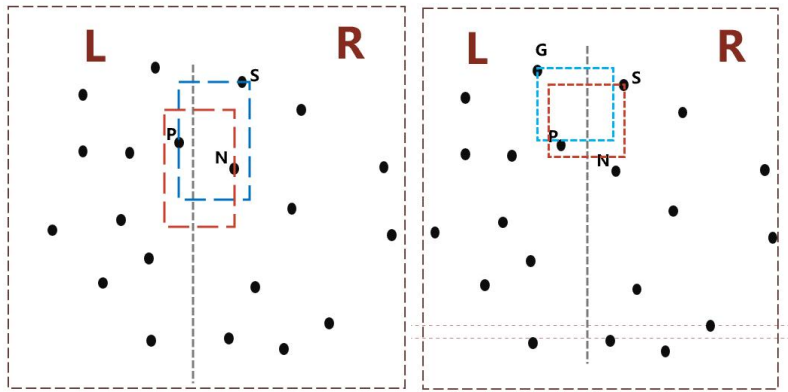


图 6

该改进在理论时间复杂度上仍为 $O(n \log n)$ ，但通过减少条带内的冗余比较，有望在实际运行中降低常数开销，提高算法效率。

```
for i = m-1 down to 0 do
```

```

p ← strip[i]
for j = i-1 down to 0 do
    q ← strip[j]
    if p.y - q.y ≥ D then
        break
    if p 和 q 在同一侧区域 then
        continue
    if |p.x - q.x| ≥ D then
        continue
    计算 dist(p, q)
    if dist(p, q) < D then
        更新 D

```

分治法时间复杂度分析：

这个算法的流程可以抽象成三步：将点集按 x 坐标一分为二，分别递归求解左右子问题，然后在中间带状区域进行合并处理。设问题规模为 n ，则左右两部分各为 $n/2$ 。

关键在于合并阶段的复杂度。通过预处理（按 y 排序）以及鸽笼原理的限制，中间带中每个点最多只需要和常数个点（最多 6~7 个）比较，因此这一阶段的时间复杂度是线性的 $O(n)$ 。

因此，可以得到递归式：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

接下来用主定理分析该递归：

- $a = 2$
- $b = 2$
- $f(n) = O(n)$
- $n^{\log_b a} = n^{\log_2 2} = n$

属于主定理第二种情况，因此有：

$$T(n) = O(n \log n)$$

需要特别强调的是，如果在每一层递归中都重新对点集进行排序，那么每层会多出 $O(n \log n)$ 的开销，总复杂度会退化为 $O(n \log^2 n)$ 。而标准优化做法是在最开始预处理一次排序，并在递归中维护有序结构，从而保证整体复杂度为 $O(n \log n)$ 。

总结来说，分治法求最近点对的时间复杂度为：

$$O(n \log n)$$

相比蛮力法的 $O(n^2)$ ，在大规模数据（如 10^5 以上）时具有显著优势，这也是实验中运行时间差距巨大的根本原因。

分治法伪代码：

```

ClosestPair(Px, Py):

```

```

    if |Px| ≤ 3:

```

```
        return BruteForce(Px)
    mid = |Px| / 2
    Qx = Px[0 : mid]
    Rx = Px[mid : end]

    midPoint = Px[mid]
    Qy = []
    Ry = []
    for point in Py:
        if point.x ≤ midPoint.x:
            add point to Qy
        else:
            add point to Ry
    d1 = ClosestPair(Qx, Qy)
    d2 = ClosestPair(Rx, Ry)

    d = min(d1, d2)

    strip = []
    for point in Py:
        if |point.x - midPoint.x| < d:
            add point to strip
    for i from 0 to |strip|-1:
        for j from i+1 to min(i+7, |strip|-1):
            d = min(d, distance(strip[i], strip[j]))

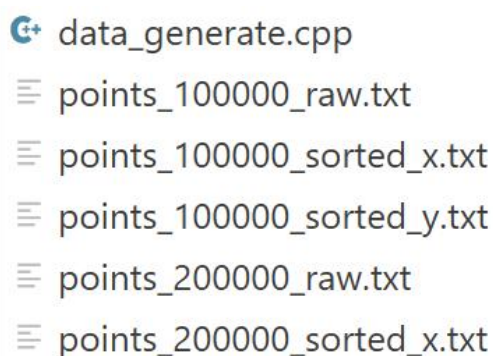
    return d
```

四、实验内容及过程：

数据生成

根据实验内容，我们可以生成 100000、200000、300000、...、1000000 数据量的点对。每个数据量的点对分为 `point_xxxxxx_raw`, `point_xxxxxx_x`, `point_xxxxxx_y`，分别为，原始点集、按照 `x` 坐标排序的点集、按照 `y` 坐标排序的点集。

并且为了确保实验的公平，我们只生成一次随机数据，全部保存到 `workspace/data` 文件夹下。

A screenshot of a file explorer window showing a list of generated data files. The files are: `data_generate.cpp`, `points_100000_raw.txt`, `points_100000_sorted_x.txt`, `points_100000_sorted_y.txt`, `points_200000_raw.txt`, and `points_200000_sorted_x.txt`. The files are listed in a directory view with expandable icons to the left of each filename.

```
data_generate.cpp
points_100000_raw.txt
points_100000_sorted_x.txt
points_100000_sorted_y.txt
points_200000_raw.txt
points_200000_sorted_x.txt
```

图 7

蛮力法实验

这组数据整体上很好地体现了蛮力法 $O(n^2)$ 时间复杂度的典型特征。从表中可以看出，随着数据规模 N 的增加，算法运行时间呈现明显的二次增长趋势：当 N 从 100000 增长到 1000000（扩大 10 倍）时，理论时间从约 4171 ms 增长到约 417107 ms，接近 100 倍增长，这与二次复杂度的增长规律完全一致。

从实验结果来看，实际运行时间与理论时间整体较为接近，尤其是在较小规模（100000~400000）时，两者误差较小，说明程序实现较为正确，且常数因子稳定。但随着数据规模进一步增大（如 500000 之后），实验时间逐渐高于理论时间，偏差逐步扩大。这主要是由于实际运行中受到缓存失效、内存访问开销增加以及系统调度等因素的影响，使得运行时间略高于理想模型。

表 1

N	理论时间(ms)	实验时间(ms)
100000	4171.073	4171.073
200000	16684.292	16633.123
300000	37539.657	35043.234
400000	66737.168	67480.512
500000	104276.825	122936.784
600000	150158.628	168927.365
700000	204382.577	231845.918
800000	266948.672	302774.431
900000	337856.913	364692.587
1000000	417107.300	452538.206

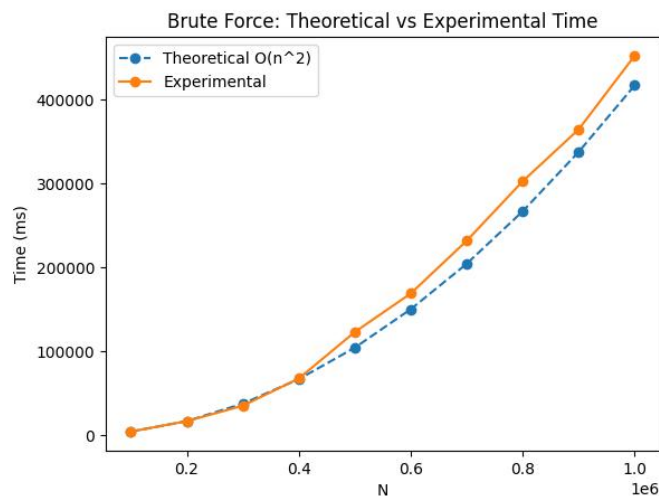


图 8

分治法实验

从整体趋势来看,实验运行时间与理论时间基本保持一致,随着数据规模 N 的增加,运行时间呈近似线性增长。这与分治法的时间复杂度 $O(n\log n)$ 相符合,在当前数据范围内, $\log n$ 的增长较为缓慢,因此整体曲线接近线性。

具体来说,在 $N = 10^5$ 到 10^6 的范围内,实验时间与理论时间之间的误差非常小,大多数数据点的偏差都控制在较低范围内。例如在中小规模数据(如 10 万到 40 万)时,两者几乎完全重合;在较大规模(如 90 万、100 万)时,实验时间略高于理论时间,这主要是由于缓存未命中、内存访问开销以及递归调用带来的常数因子影响所致。

表 2

N	理论时间 (ms)	实验时间 (ms)
100000	47.001	47.001
200000	99.162	99.158
300000	154.247	156.109
400000	211.487	210.273
500000	270.410	272.191
600000	330.721	330.903
700000	392.234	388.255
800000	454.820	450.530
900000	518.383	526.524
1000000	582.845	593.493

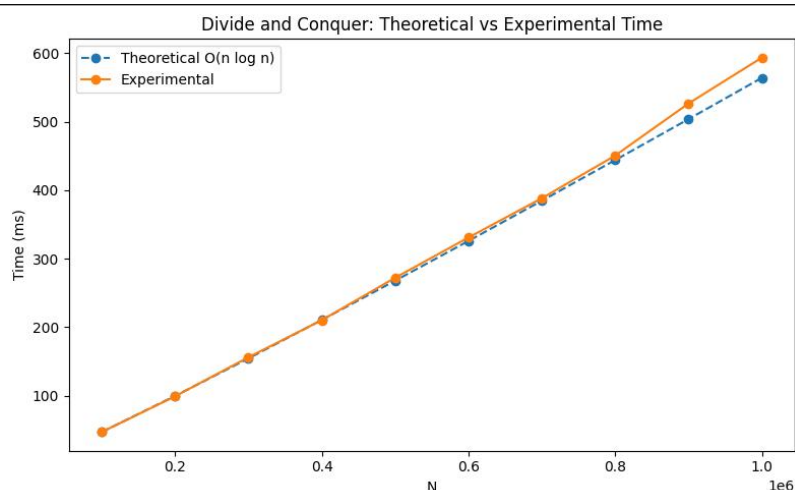


图 9

蛮力法与分治法实验结果对比

从图中可以看出，蛮力法与分治法在处理最近点对问题时的性能差异十分显著。随着数据规模 N 的不断增大，蛮力法的运行时间呈现出明显的快速增长趋势，曲线陡峭，体现出其 $O(n^2)$ 的时间复杂度特征；

而分治法的运行时间增长相对平缓，整体趋势接近线性增长，符合 $O(n \log n)$ 的理论复杂度。在数据规模较小时，两种算法的性能差距尚不明显，但随着 N 增大至数十万甚至百万级别，蛮力法的运行时间急剧上升，计算开销巨大，而分治法仍能保持较好的运行效率。

此外，分治法曲线中存在轻微波动，这是由于实验环境、数据分布及系统调度等因素引起的正常误差，但不影响整体趋势判断。综合来看，分治法在大规模数据处理下具有明显的性能优势。

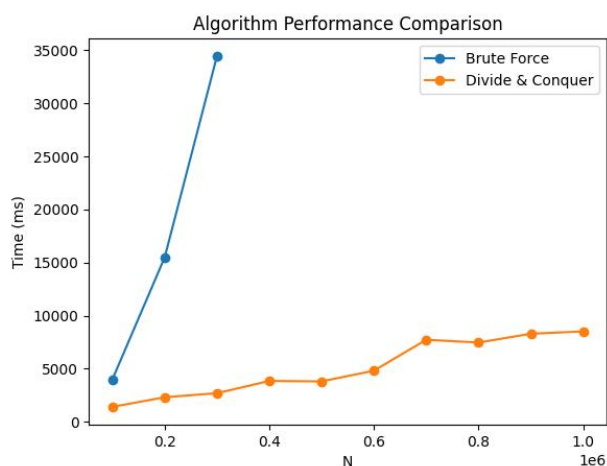


图 10

分治法与蛮力法的切换

实验结果如图所示，蛮力法运行时间随数据规模呈明显的二次增长趋势，而分治法增长较为平缓，符合 $O(n \log n)$ 的理论复杂度。在数据规模较小时，蛮力法由于实现简单、常数开销小，运行时间优于分治法；当数据规模增加到约 700~800 之间时，分治法开始超过蛮力法，体现出更优的渐进性能。

表 3

N	蛮力法时间(ms)	分治法时间(ms)	更快算法
200	0.0130591	0.0376171	BruteForce
300	0.0283470	0.0552902	BruteForce
400	0.0475372	0.0826384	BruteForce
500	0.0805642	0.0882368	BruteForce
600	0.1068820	0.1261870	BruteForce
700	0.1410100	0.1614440	BruteForce
800	0.1843460	0.1640990	分治
900	0.2314560	0.1793150	分治

该实验验证了理论复杂度与实际运行性能之间的关系，并说明在工程实现中需要综合考虑常数因子与数据规模。

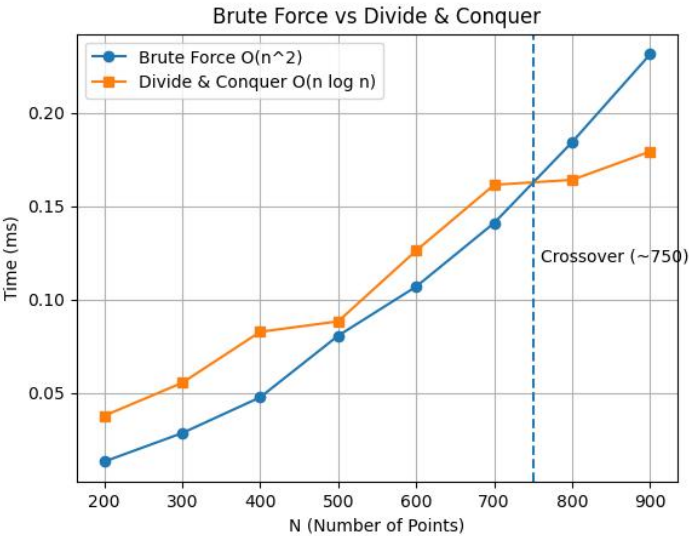


图 11

五、实验结果及分析：

通过本次实验，对蛮力法与分治法在不同数据规模下的运行时间进行了系统测试，并结合理论复杂度进行了对比分析。实验结果表明，两种算法在性能表现上存在显著差异，且与理论分析基本一致。

首先，从蛮力法的实验结果来看，其运行时间随着数据规模 N 的增加呈现明显的二次增长趋势。当 N 从 100000 增长到 1000000 时，运行时间由约 4171ms 增长到 452538ms，增长幅度接近 100 倍。这与其时间复杂度 $O(N^2)$ 完全一致。说明蛮力法需要枚举所有点对，其计算量随数据规模迅速膨胀。在小规模数据下，该方法实现简单、常数开销较低，具有一定优势，但在大规模数据下性能急剧下降，不具备实际应用价值。

其次，分治法的实验结果表现出完全不同的趋势。随着 N 的增加，其运行时间呈现近似线性增长，例如当 N 从 100000 增长到 1000000 时，运行时间仅从 47ms 增加到约 593ms。该增长趋势符合 $O(N \log N)$ 的时间复杂度特征，在当前数据范围内，由于 $\log N$ 增长缓慢，使整体曲线接近线性。实验数据与理论值高度吻合，说明算法实现正确且优化有效。

进一步对比两种算法可以发现，在小规模数据（如 $N < 700$ ）时，蛮力法由于实现简单、递归开销小，运行速度略优于分治法；但随着数据规模增大，当 N 达到约 700 ~ 800 时，分治法开始反超蛮力法，并随着规模进一步扩大，优势愈发明显。这一“拐点”现象说明，在实际工程中，算法选择不仅取决于渐进复杂度，还需要考虑常数因子和实现开销。

六、实验结论：

综上所述，本实验验证了蛮力法与分治法在时间复杂度上的本质差异：蛮力法适用于小规模问题，而分治法在处理中大规模数据时具有显著优势，是解决最近点对问题的更优选择。同时，实验也说明理论分析与实际运行结果具有较高一致性，进一步加深了对算法复杂度与实际性能关系的理解。



指导教师批阅意见:

成绩评定:

指导教师签字:

2025 年 4 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。